

PHOENICIAN CAPITAL · INTERNAL METHOD PAPER

Local LLM Internet Engine

Full method

Exactly what was researched, decided, built, tested, and shipped in `local-engine-internet` — the search and fetch layer that lets a local DeepSeek Flash Brain use the internet when it needs to, without inventing facts or URLs.

Repository: github.com/phoenician-capital/local-engine-internet

Commit described: 96315a1 on main

Date: 8 September 2026

Audience: anyone who must understand, operate, or wire this engine

Scope: research of every phoenician-capital repo, the Mode 1 / Mode 2 split, every module, every contract, every fail rule, every test

Contents

1. What was done, in order	
2. The problem this repo exists to solve	
3. The one distinction the whole engine is built around	
4. Research method – every company repo	
5. Findings that constrained the design	
6. Topology decision	
7. What was reused versus invented	
8. Repository layout	
9. Data contracts	
10. Search method	
11. Fetch method	
12. Tools the model is allowed to call	
13. Mode 2 agentic loop – step by step	
14. HTTP API	
15. Fail-closed policy	
16. Configuration and operations	
17. Wiring other repos (not done in this repo)	
18. Tests and verification	
19. What this engine deliberately does not do	
20. File-by-file method index	
21. How to run it	

1. What was done, in order

This is the full sequence, not a summary of “we built a search API.” Each step exists because the previous one produced a constraint.

1.1 Create the repository

The GitHub repository `phoenician-capital/local-engine-internet` was created under the organization so every team can use it later. Owner is the org. An initial README stated the intent: a dedicated internet layer for local LLMs.

1.2 Clone every phoenician-capital project locally

Every organization repository was cloned into `org-repos/` (gitignored, never committed). The point was research, not a monorepo. Those clones were read in place. Nothing from them was copied into git except the *ported logic*, rewritten into this service’s own packages.

1.3 Extreme-depth LLM audit (first pass)

Every cloned project was inspected for LLM use. Result: `.cursor/research/llm-audit.md` and always-on Cursor rules:

- `.cursor/rules/00-project.mdc` — what this repo is for
- `.cursor/rules/01-company-llm.mdc` — which repo uses which model
- `.cursor/rules/02-search-today.mdc` — how the company searches today
- `.cursor/rules/03-integration.mdc` — how this engine must speak to callers

1.4 Second pass – every bit of LLM logic

Line-level catalogs were written so we would not invent a protocol the apps do not already speak:

- `logic-01-phoenician-intelligence.md` — every PI wrapper, retry, monkey-patch, section route
- `logic-02-portfolio.md` — Claude vs DeepSeek clients, debate, universe (tools deleted)
- `logic-03-screening-capiq-earnings.md` — screening factory, CapIQ desks, Earnings tracker
- `logic-04-router-and-light.md` — ai-router tool loop, Linker, investor-portal, dcf-app, confirmed no-LLM repos

1.5 Third pass – usage, fail policy, external APIs

- `logic-05-third-pass-usage.md` — purpose, data flow, fail-open vs fail-closed per product
- `logic-06-external-apis.md` — SerpAPI, Tavily, Google CSE, FMP, CapIQ, AlphaSense, Vertex, embeddings

1.6 Engine bible and surgical addendum

Those catalogs were distilled into a build spec, not more notes:

- `engine-bible.md` — job, insertion points, contracts, fail rules, env, wiring checklist
- `engine-surgical.md` — SerpAPI extra fields, Earnings fetch ladder, JS IR labels, SEC UA, deny lists, citation shapes, prompt budgets

Where the two disagree on fetch or SERP details, the surgical file wins.

1.7 Plan, then implement

A build plan was written around one law: do not confuse pipeline-directed internet (code already decided to search) with LLM-directed internet (the model decides mid-generation). Then the plan was implemented in this repo: scaffold, search providers, fetch ladder, extract, tools, agent loop, HTTP API, tests, README. The plan file itself was not edited.

1.8 Verify, commit, push

43 unit tests passed (pytest + respx, no live keys required). The work was committed as 96315a1 and pushed to origin/main. Secrets and org-repos/ were not committed.

2. The problem this repo exists to solve

The company is moving inference to a local DeepSeek V4 Flash Brain on vLLM, reached through ai-router on port 8080. A local model has weights and a context window. It does not have a browser, a SERP, or a PDF parser.

Today, “the model looked it up” is almost always a *vendor server tool*:

- Anthropic web_search_20250305 — DD writing, screening analyst/memo, portfolio debate, CapIQ chat/auditor
- Gemini GoogleSearch — PI call_gemini_web, Earnings news, dcf--app
- Perplexity Sonar — PI prefetch, red flags, TAM, technicals, H2H, screening news
- DeepSeek cloud /responses web_search — PI cheap workflow only
- Z.ai web_search — CapIQ GLM desks
- SerpAPI — Earnings calendar/extract, PI Trustpilot/Reddit/IR, ai-router snippets
- Tavily — ai-router only

ai-router already has a thin web_search tool (SerpAPI + Tavily snippets, max 3 rounds, no page fetch, no ranking, no citations). If no keys are set, it tells the model to answer from its own knowledge. Research callers treat that as a bug.

Setting DEEPSEEK_BASE_URL to the router does *not* enable search. Callers must send phoenician_tools. In PI, that base URL is still hardcoded to <https://api.deepseek.com>.

This repo is internet only. It is not a second inference proxy. Load-balancing, Brain auth, and GPU routing stay in ai-router. Prompts and workflows stay in each product.

3. The one distinction the whole engine is built around

There are two ways the organization touches the internet. Mixing them produces the wrong product: either the model is forced to search when code already did, or code is asked to “be smart” when the model should decide.

	Mode 1 – pipeline-directed	Mode 2 – LLM-directed
Who decides	Application code already knows it needs the web	The model decides mid-answer
LLM's job	Judge or extract after search (Earnings ranker)	Choose query, choose URL, write with sources
Today	SerpAPI from Python/JS, then fetch in Earnings	Vendor server tools (Claude / Gemini / Perplexity / DeepSeek cloud)
This engine	POST /v1/search, /v1/fetch, /v1/research, GET /search	POST /v1/chat/completions with an agentic tool loop
Typical callers	Earnings, EP smart_finder, PI Trustpilot/Reddit/IR	PI cheap path, portfolio DeepSeek, screening / CapIQ local desks

Same search providers and fetch ladder underneath. Different entry points. Never mixed inside one handler.

Mode 2 is the product. Local Flash has no vendor web tool. Mode 1 is exposed so pipelines that already decided to search do not have to pretend they are an agent.

4. Research method – every company repo

Clones were classified into heavy AI, light AI, and no LLM. That map is what decided who this engine must serve first.

4.1 Heavy AI (must keep working after the local switch)

Repo	Inference today	Search / grounding today
phoenician-intelligence	Claude (PI_DD / PI_CHATBOT), Gemini, OpenAI, DeepSeek v4-flash, Perplexity, optional Grok	Anthropic web_search, Gemini GoogleSearch, DeepSeek web_search, Perplexity Sonar, Vertex/Gemini/AlphaSense RAG
phoenician-portfolio	Claude Sonnet/Opus/Fable; optional DeepSeek v4-flash	Anthropic web in debate; DeepSeek universe has <i>no</i> web search (tools deleted)
capiq-screen-agent	Claude, DeepSeek, GLM/Z.ai, OpenRouter Llama, OpenAI judges	Anthropic web, Z.ai web, Managed Memory/Dreams
screening-engine	Claude, OpenAI, Gemini, Perplexity	Anthropic web, Perplexity ingest, OpenAI embeddings + pgvector
Earnings_tracker	DeepSeek v4-pro / chat, Gemini	SerpAPI + Gemini Google Search grounding

4.2 Light AI

Repo	Usage
investor-portal-backend	OpenAI gpt-4o-mini / gpt-4o admin PDF extraction only
Linker	Optional Claude qualitative 0-40 score
dcf-app	Gemini flash + Google Search, client-side key
phoenician-intelligence-backend	OpenAI TTS proxy + admin Claude for logs; no report generation
phoenician-intelligence-frontend	OpenAI TTS + chatbot UI; no report LLM

4.3 No LLM

pi-global, phoenician-frontend-kit, phoenician-mail-sender, Factsheet-Automation, phoenician-capital-website, investor-portal-web, investor-portal-mobile, investor-portal-backend-archive, empty dcf-app.

4.4 Shared keys already in the org

ANTHROPIC_API_KEY, PI_DD, PI_CHATBOT, OPENAI_API_KEY, DEEPSEEK_API_KEY, DEEPSEEK_BASE_URL, PERPLEXITY_API_KEY, GEMINI_API_KEY, SERPAPI_KEY, TAVILY_API_KEY, AWS secret phoenician-ai-api-keys.

This engine must not require Anthropic, Perplexity, Gemini, or PI_DD to function. Those are what we are removing from the search path. It may use SerpAPI / Brave / Tavily / CSE / SearXNG as backends until a local index exists.

5. Findings that constrained the design

1. **Almost nothing talks to ai-router.** Apps call cloud SDKs. The intended hook is DEEPSEEK_BASE_URL.
2. **Local Brain is Flash only.** Earnings and some PI paths call deepseek-v4-pro. Do not silently route Pro to a Flash-only Brain.
3. **Web search is vendor-native.** Local Flash has none of those tools unless this engine (or a later router client) provides them.
4. **Cheap PI path is a monkey-patch** of Claude/Gemini/Perplexity onto DeepSeek. DCF streaming and Gemini File RAG are not patched.
5. **Fail policies disagree.** Screening memo can fail-open to templates. PI Section 8 DCF raises. Router search fail-opens to memory. The engine default is fail-closed for research.
6. **Snippets-only will fail DD, CapIQ research, and Earnings.** Fetch + extract had to ship on day one.
7. **The company already has a fetch engine** in Earnings (httpx → curl_cffi → Playwright → PDF/SEC). Port it. Do not invent a new one.
8. **SerpAPI is richer than title/snippet/link.** EP calendar quality depends on answer_box, knowledge_graph, and the first two related_questions. The router throws those away. This engine keeps them.
9. **vLLM Flash must be launched with tool parsers** or it will not emit structured tool_calls. Streaming tool calls leak DSML into arguments.
0. **PI writers parse two citation shapes.** A replacement that emits only one of them loses hyperlinks.
1. **Deny lists are per-mode.** Earnings fetches SEC. PI IR discovery must not treat sec.gov as the company's IR site.

6. Topology decision

The engine sits **in front of** ai-router on port 8090.

```

Company apps
├── OpenAI chat-completions body
│   (optional phoenician_tools / phoenician_web)
└──
    ▼
local-engine-internet :8090
├── Mode 2 /v1/chat/completions → tool loop → UPSTREAM_LLM_BASE_URL (ai-router :8080) → vLLM
Flash
├── Mode 1 /v1/search /v1/fetch /v1/research
├── Compat GET /search (SerpAPI-shaped JSON)
│   ├── search providers (Brave, SerpAPI, Tavily, SearXNG, optional CSE)
│   └── fetch ladder (httpx → curl_cffi → Playwright → PDF / SEC)

```

Apps that only change DEEPSEEK_BASE_URL to this engine get “search when needed” because the default policy is auto and the three tools are injected. That is a deliberate product difference from ai-router, which is opt-in via phoenician_tools. Pass-through without tools is phoenician_web.policy: "off".

ai-router is unchanged in this work. Its run_web_search can later become a thin HTTP client to POST /v1/search. That wiring is documented, not executed here.

7. What was reused versus invented

The method was “port proven org code, then add only what no repo has.”

Need	Source in org-repos	This repo
Tool loop, phoenician_tools, trace + summed usage	ai-router orchestrator.py, http.py, main.py	engine/agent/loop.py, upstream.py
Settings / auth / health style	ai-router config.py, auth.py	engine/config.py, auth.py
SerpAPI full-field parse + 0.4s pace + site: misses	portfolio EP smart_finder.py :570-646	engine/search/serpapi.py
Tavily client	ai-router web_search.py	engine/search/tavily.py
Query hygiene (legal suffix, filetype:pdf, site:)	Earnings summary/search.py	engine/search/query.py
Fetch ladder + FetchMode	Earnings fetch.py, config.py	engine/fetch/ladder.py
HTML → text, PDF-link ranking	Earnings html_cleaner.py	engine/fetch/extract.py + urlutil
PDF chain	Earnings pdf_parser.py	engine/fetch/pdf.py
SEC exhibits + UA	Earnings sec_handler.py	engine/fetch/sec.py
JS IR labels (EN/JP/PL)	Earnings constants.py	engine/fetch/js_interact.py
Fetch cache TTL / negative cache	Earnings fetch_cache.py	engine/fetch/cache.py
WAF markers	PI serpapi_helper.py	engine/fetch/policy.py
Deny lists per mode	PI prefetch_pipeline.py, company_pdf_downloads	engine/fetch/policy.py
When-to-search + never fabricate URLs	PI call_deepseek.py _WEB_WRITING_SYSTEM_MESSAGE	engine/agent/prompts.py
force_web_search → required	PI call_deepseek.py	engine/agent/loop.py policy
Citation blocks	PI web_search.py :70-93	engine/agent/citations.py
150k web context budget	PI config.py	WEB_CONTEXT_BUDGET_CHARS
Google CSE official site	PI prefetch_pipeline.py	engine/search/google_cse.py

New (no org precedent): Brave provider, SearXNG slot, Trafilatura layer, DSML recovery, per-request budgets, /capabilities Brain canary, SerpAPI-compatible GET /search, fail-closed strings that never say “answer from your own knowledge.”

The Earnings LLM ranker, SEC DeepSeek exhibit fallback, and 37-query explosions were *not* ported. Those are product judgment, not internet infrastructure.

8. Repository layout

```
engine/  
  main.py          FastAPI app, lifespan (httpx client, cache, canary)  
  config.py        every env var, FetchMode / DomainMode / WebPolicy  
  auth.py          Bearer ENGINE_API_KEY; GET /search also accepts api_key=  
  errors.py        SearchUnavailable, RequiredSearchFailed, fail-closed strings  
  runtime.py       process-wide http client, cache, capabilities dict  
  metrics.py       Prometheus counters  
  budget.py        per-request search / fetch / char budgets  
  search/          Hit, providers, merge, query hygiene  
  fetch/           ladder, extract, pdf, sec, js_interact, cache, policy  
  tools/           OpenAI schemas, execute, DSML fallback  
  agent/           loop, upstream, prompts, citations  
  api/             chat, primitives, compat, health  
tests/             pytest + respx (43 tests)  
scripts/smoke.py   live check against a real router + one search key  
.env.example       Dockerfile docker-compose.yml README.md  
.cursor/research/  bible, surgical, audits (committed)  
org-repos/         local clones only (gitignored)
```

9. Data contracts

9.1 Hit

Defined in engine/search/base.py. Sources stay labeled. The model weighs them; the engine does not pick a “best” snippet.

```
{
  "title": str,
  "url": str,
  "snippet": str,
  "source": "serpapi" | "brave" | "tavily" | "searxng" | "google_cse",
  "kind": "organic" | "answer_box" | "knowledge_graph" | "related_question",
  "position": int,
  "date": str | null,
  "query": str
}
```

Dedup key: canonical URL (scheme + host without www + path without trailing slash), then snippet[:120].

9.2 Page

Defined in engine/fetch/ladder.py.

```
{
  "url": str, "final_url": str, "title": str, "text": str,
  "content_type": str, "is_pdf": bool,
  "tier": "httpx" | "curl_cffi" | "playwright" | "sec" | "cache",
  "ok": bool, "error": str | null
}
```

9.3 Chat request extras (stripped before upstream)

- phoenician_tools — subset of web_search, fetch_url, search_and_read. Omitted means all three (engine default). Empty list means no loop.
- phoenician_web — { policy, max_search_uses, max_fetches, max_rounds, fetch_mode, domain_mode, providers, web_context_budget_chars }

Caller Authorization is dropped. Upstream receives UPSTREAM_API_KEY. X-Priority is forwarded for ai-router.

9.4 Chat response extras

- phoenician_tool_trace — [{tool, arguments}, ...] identical idea to ai-router
- phoenician_usage_total — summed prompt/completion/total across every upstream round
- phoenician_sources — URLs actually returned by search or fetch
- phoenician_citations_block — both PI citation forms
- phoenician_search_failed — true if a provider raised

10. Search method

Providers are enabled only by the presence of keys. Adding a key later changes nothing else. Per-provider timeout is 15 seconds. One failure is skipped and recorded. If every attempted provider fails, or none are configured, the engine raises `SearchUnavailable` with a fail-closed string — it does not invent results and does not tell the model to use training data.

Default fan-out order when the caller does not pin providers: Brave, SerpAPI, Tavily, SearXNG (whichever are configured). Google CSE is *pin-only* (official-website fallback), not part of the default fan-out even if keys exist.

10.1 SerpAPI – `engine/search/serpapi.py`

GET `https://serpapi.com/search` with `engine=google`, `hl=en`, `gl=us`. Key from `SERPAPI_KEY` or `SERPAPI_API_KEY`.

Parser (EP `smart_finder`):

- `organic`: title, snippet, link, position, date; skip duplicate URL or `snippet[:120]`
- `answer_box`: answer or snippet + link, kind `answer_box`
- `knowledge_graph`: title + description + attribute values, kind `knowledge_graph`
- `related_questions`: first **two** only

Recency maps to `tbs=qdr:d|w|m|y`. Consecutive queries are paced 0.4s (disable in tests via `SERPAPI_PACE_SECONDS=0`). Site-restricted empty responses are a valid empty set, not a burned failure — that pacing/budget rule lives in EP's multi-query loop; a single `/v1/search` just returns empty hits from that provider.

10.2 Brave – `engine/search/brave.py`

GET `https://api.search.brave.com/res/v1/web/search` with header `X-Subscription-Token`. Hits from `web.results[]` (title, url, description, age). Recency: `freshness=pd|pw|pm|py`.

10.3 Tavily – `engine/search/tavily.py`

Port of the router client. POST `https://api.tavily.com/search` with `api_key`, `query`, `max_results`, optional `days`.

10.4 SearXNG – `engine/search/searxng.py`

GET `{SEARXNG_URL}/search?q=&format=json&categories=general`. Optional compose profile `searxng`.

10.5 Google CSE – `engine/search/google_cse.py`

PI official-website fallback. GET `https://www.googleapis.com/customsearch/v1?key=&cx=&q=&num=` (num capped at 10, timeout at least 20s). Items: link, title, snippet. Same `GOOGLE_API_KEY` also unlocks Gemini in other apps — this module is CSE only.

10.6 Merge – `engine/search/merge.py`

1. Resolve provider list (pin or default fan-out $\cap$ configured).
2. `asyncio.gather` all providers.
3. Collect hits and errors. Keep raw SerpAPI extras for the compat endpoint.

4. Apply domain_mode deny list.
5. Dedup. Keep up to num_results organics plus all specials (answer box / KG / related).
6. Format for the model as labeled blocks: [serpapi:organic], [brave:organic], etc.

on_empty=error raises if there are zero hits after a successful provider call. Default is empty (scrapers).

10.7 Query hygiene – engine/search/query.py

Port of Earnings build_simple_queries:

- Strip legal suffixes (PLC, Inc, Ltd, Corp, S.A., ASA, ...)
- site:{host} and filetype:pdf builders
- Aggressive PDF-oriented query list: quoted company + quarter/year, ticker, IR domain, q4cdn/mziq, cap 15, deduped

The engine does not explode into 37 templates. Judgment stays with the caller (ranker or model).

10.8 Domain modes – engine/fetch/policy.py

Mode	Deny list	Why
general_research	facebook, instagram, twitter, x.com, tiktok	Plan: empty except spam/social
ir_discovery	PI aggregator list + IR PDF blocklist including sec.gov / edgar.sec.gov, nasdaq, nyse, asx, yahoo, bloomberg, ...	Never treat an aggregator or EDGAR as “the company’s IR site”
earnings_doc	social only – SEC allowed	Earnings does fetch EDGAR exhibits

Fetch also always skips facebook / twitter / instagram regardless of mode (Earnings behaviour).

11. Fetch method

Canonical source: Earnings tracker/summary/fetch.py, rewritten async-first for FastAPI. `FETCH_MODE` is `full` (all tiers), `no_browser` (`httpx` + `curl_cffi`), or `httpx_only` (Lambda-style, no Chromium).

11.1 Ladder order – engine/fetch/ladder.py

1. Reject empty URL, social hosts, domain-mode deny list.
2. Return process-wide cache hit if present (success TTL 1800s, failure TTL 120s, max 256 entries).
3. Direct PDF (non-SEC): `httpx` with document headers and same-origin Referer; on WAF statuses {401,403,406,409,429,503} escalate to `curl_cffi` impersonate chrome; then Playwright request stack after warming the origin.
4. IR-looking URL (`ir.` or “investor” in the URL) and `mode=full`: Playwright first, because static HTML is often boilerplate.
5. Otherwise `httpx` with Chrome-like headers, `Accept-Language`, `Sec-Fetch-*`, `gzip/br`, and age-gate cookies (`age_verified`, `over18`, `legal_age`, `is_over_18`).
6. If the body looks like a WAF challenge (PI markers in the first 5 kB: “please verify you are a human”, Cloudflare, Sucuri, Distil “pardon the interruption”, “are you a robot”, “403 forbidden”) treat it as a block and escalate.
7. SEC wrapper URLs (`sec.gov`, or `Yahoo cdn.yahoofinance.com` + `sec-filings`) go through exhibit discovery.
8. Thin HTML (< 400 chars) escalates to Playwright when `mode=full`.
9. Extract, cap (8k HTML / 25k PDF unless the caller set `max_chars`), cache, return Page.

PDF detection: magic bytes `%PDF`, or content-type containing `pdf` unless the body is HTML disguised as PDF.

Limits copied from Earnings: document timeout 20s, max PDF 25 MB, headless render 15s, thin threshold 400 chars.

11.2 Extract – engine/fetch/extract.py

Trafilatura first (markdown, links and tables, multilingual). If it yields nothing, Earnings `html_to_text`: strip `script/style/svg/head`; `br/p/div/li/tr/h*` → newline; unescape `nbsp/amp/lt/gt/quot/euro/pound/yen`; drop `Cookie/Privacy/Accept Cookies` chrome. Title from Trafilatura metadata or `<title>`. Canonical from metadata or `rel=canonical`.

11.3 PDF chain – engine/fetch/pdf.py

`pdfplumber` → `PyMuPDF (fitz)` → `pypdf (strict=False)` → `pdftotext/poppler` → raw regex of readable strings. First parser that yields usable text wins. If all fail, placeholder: “This document is a PDF report available at: {url}” — never a hallucinated transcript.

11.4 SEC – engine/fetch/sec.py

Compliant User-Agent is required or EDGAR 429s. Default `SEC_EDGAR_USER_AGENT` identifies LocalEngineInternet + `phoeniciancapital.com`. Wrapper pages are not the earnings release. The handler:

- Regex-extracts EX-99.1 / EX-99.2 / exhibit hrefs; prefers PDF then EX-99.1
- On Yahoo wrappers, ranks sibling `.htm/.pdf` links by “earnings release”, “presentation”, ...

- Fetches up to five exhibits with the SEC UA
- Falls back to wrapper body extraction and embedded <DOCUMENT><TYPE>EX-99.1 blocks

The Earnings DeepSeek exhibit ranker was not ported. This is fetch, not judgment.

11.5 JS IR interaction – engine/fetch/js_interact.py

Matched by accessible name / role, never by company. Ported constants:

- Expanders: load/show/view more, もっと見る, 一覧, すべて, さらに表示, 過去
- Tabs: [role=tab], Bootstrap/ARIA accordions
- Results labels: financial results, 決算, 四半期, 有価証券報告書, 決算短信, 適時開示, raporty okresowe
- Hidden URL attrs: data-url, data-pdf, data-file, ...
- Doc hosts without .pdf: xj-storage.jp, irpocket, q4cdn, mziq, cloudfront /documents/
- Wait for spinners: .loading, [aria-busy=true], ...
- Age-gate buttons: “I am 18”, Enter, Confirm, ...
- Iframe snapshots (Investis/EQS RNS widgets) with links rewritten absolute against the frame URL

Snapshots after each state change are concatenated so a tab click that replaces the DOM does not lose earlier PDF links.

12. Tools the model is allowed to call

OpenAI function-calling schemas in `engine/tools/schemas.py`. vLLM Flash must emit `tool_calls` in that shape.

12.1 web_search

Arguments: `query` (required), `num_results`, `recency_days`. Description tells the model results are raw and unlabeled-by-quality — weigh them itself. Output is labeled text, not JSON hidden from the model.

12.2 fetch_url

Arguments: `url` (absolute http/https), `max_chars`. Output always starts with `[PUBLIC_URL: ...]` so PI's URL rule can be obeyed. Only pass URLs search returned or the user gave.

12.3 search_and_read

Arguments: `query`, `top_n` (default 3). One round: search, then fetch organic URLs. Exists because Flash has a low useful tool-round budget; the router's max of 3 was too low once fetch is a tool, so the engine default is 6 rounds and this composite saves one hop.

12.4 Dispatch – `engine/tools/execute.py`

`ToolExecutor` holds the budget, domain mode, fetch mode, optional provider pin, and the accumulating sources list. Over-budget calls return an ERROR string. Invalid URLs return an ERROR string. Unknown tool names return an ERROR string. Nothing invents a page.

12.5 DSML fallback – `engine/tools/dsml_fallback.py`

Known Flash/vLLM failure: `tool_calls` is empty but content contains markup. Recovered forms:

- Fullwidth-bar DSML: `< | DSML | invoke name="web_search">` with parameter tags (ASCII | also accepted)
- Open invoke without a closer (start-token omission at long context)
- DeepSeek tokenizer blocks: `tool_calls_begin` / `tool_sep` + JSON fence

Recovered calls are written back onto the assistant message and the markup is stripped so a later reader does not treat DSML as the answer.

13. Mode 2 agentic loop – step by step

File: engine/agent/loop.py. Non-streaming only.

1. Strip phoenician_tools and phoenician_web.
2. If policy is off or tools list is empty: single post_completion to ai-router. Return. No guidance injected.
3. Otherwise inject the three (or requested) OpenAI tool schemas. Default tool_choice=auto.
4. Prepend PI web system guidance (or merge into the caller’s existing system message): use tools when facts may have changed; prioritize official filings; **never fabricate URLs**; if a tool result starts with ERROR, treat it as a hard failure; no “Sources” appendix; no preamble.
5. POST to UPSTREAM_LLM_BASE_URL (ai-router), timeout 300s, forward X-Priority, send upstream key.
6. Read tool_calls. If empty, run DSML recovery.
7. If still empty and policy is required and we have not already re-asked: append a user nudge (“You must call the web_search tool before answering. Search now.”), set tool_choice forced on web_search, POST once more.
8. If still empty after that re-ask (or search never succeeded): raise RequiredSearchFailed → HTTP 424.
9. Execute every tool call of the round in parallel (asyncio.gather). Enforce max_search_uses / max_fetches.
0. Append role: tool messages. Compact oldest tool outputs first if the running total exceeds WEB_CONTEXT_BUDGET_CHARS (150k). Compacted stubs keep the URLs.
1. Repeat up to MAX_TOOL_ROUNDS (default 6).
2. Collect every URL actually returned into phoenician_sources. Build both citation blocks. Return the last upstream JSON plus extras.

The model is told it may hyperlink only those URLs. A broken link is worse than no link. That sentence is from PI’s production system message, not invented here.

13.1 Policies mapped from PI

phoenician_web.policy	PI equivalent	Behaviour
auto (default)	writers, allow_offline_fallback=True	Model may answer without searching
required	force_web_search=True	Forced re-ask, then 424 if no successful search
off	plain completion	No tools, no guidance, stream allowed

13.2 Citations – engine/agent/citations.py

Both PI prefetch shapes, so section writers keep hyperlinks:

```
Citations (use these URLs for hyperlinks):
[SOURCE: Web Citation 1]
[PUBLIC_URL: https://...]

Detailed Sources (use these URLs for hyperlinks):
[SOURCE: {title} (Date: {date})]
[PUBLIC_URL: https://...]
```

14. HTTP API

FastAPI app in engine/main.py. Health is open. Everything else requires ENGINE_API_KEY when set (unset = open, same as the router). GET /search also accepts ?api_key= so scrapers can drop in a SerpAPI-shaped URL.

Method	Path	Mode	Purpose
POST	/v1/chat/completions and /chat/completions	2	Agentic loop or pass-through
POST	/v1/search	1	Hits only
POST	/v1/fetch	1	One Page
POST	/v1/research	1	Search then fetch top N
GET	/search?q=&engine=google&num=	1	SerpAPI-shaped organic_results + extras
GET	/health	–	status, upstream, providers, fetch_mode
GET	/metrics	–	Prometheus
GET	/capabilities	–	Brain canary, provider map, required vLLM flags

14.1 Stream rule

stream:true plus tools / default auto loop → HTTP 400, same sentence as ai-router. Stream is proxied only when policy is off (or tools explicitly empty). Final-answer SSE after the loop is phase 2, not built.

14.2 Compat shape

GET /search pins SerpAPI when that key exists so answer_box / knowledge_graph survive for Earnings/EP. Otherwise it synthesizes the same keys from merged Hits. Scrapers already parse organic_results[].title|snippet|link|position.

14.3 Upstream errors

Brain down or HTTP error from ai-router → 502. No cloud LLM fallback. That is intentional and matches the router's local-only law.

14.4 Startup canary

Unless RUN_CANARY_ON_STARTUP=false, lifespan sends a tiny tool-forced prompt to upstream. If no tool_calls (and no recoverable DSML), it logs the four required vLLM flags. GET /capabilities reports brain_tool_calls_supported. Treat the engine as not done on a real box until that is true.

15. Fail-closed policy

The router string *“Answer from your own knowledge instead, and say so.”* is never emitted. Tests assert the substring `own knowledge` is absent.

Situation	What the caller sees
No provider configured	Tool: ERROR: web_search unavailable (set SERPAPI_KEY / BRAVE_API_KEY / TAVILY_API_KEY / SEARXNG_URL). Do not present unverified facts as sourced. Mode 1: 502
Every provider HTTP-fails	Same ERROR / 502
Search or fetch over budget	ERROR: {kind} budget exhausted (limit=N). ...
policy=required and no successful search	HTTP 424 Failed Dependency
Successful provider, zero hits	“No results” (not a failure). Mode 1 can set <code>on_empty=error</code>
Fetch social / denied host	Page.ok=false with an error; not invented text
All PDF parsers fail	Placeholder pointing at the real URL

Callers already disagree (empty vs raise vs degrade). The engine exposes the flag; it does not pick one globally for Mode 1. Mode 2 research default is error.

16. Configuration and operations

Variable	Default	Meaning
ENGINE_HOST / ENGINE_PORT	0.0.0.0 / 8090	Bind
ENGINE_API_KEY	empty	Incoming auth; alias SEARCH_API_KEY
UPSTREAM_LLM_BASE_URL	http://127.0.0.1:8080	ai-router
UPSTREAM_API_KEY	empty	Forwarded Bearer; alias ROUTER_API_KEY
UPSTREAM_MODEL	deepseek-v4-flash	Default model field
SERPAPI_KEY / SERPAPI_API_KEY		Google SERP
BRAVE_API_KEY		Brave Search
TAVILY_API_KEY		Tavily
SEARXNG_URL		Self-hosted search
GOOGLE_API_KEY + GOOGLE_SEARCH_ENGINE_ID		CSE, pin-only
FETCH_MODE	full	full no_browser httpx_only
MAX_TOOL_ROUNDS	6	Router's 3 is too low with fetch
MAX_SEARCH_USES / MAX_FETCHES	8 / 8	Per request
WEB_CONTEXT_BUDGET_CHARS	150000	PI per-section web inject
MAX_CHARS_PER_DOC / PDF	8000 / 25000	Per document
MAX_TOTAL_DOC_CHARS	50000	Earnings composite cap (config present)
MAX_PDF_BYTES	25 MB	Skip huge files
THIN_HTML_CHAR_THRESHOLD	400	Escalate to Playwright
DOC_FETCH_TIMEOUT	20s	Page fetch
SEARCH_PROVIDER_TIMEOUT	15s	Per provider
REQUEST_TIMEOUT	300s	Upstream LLM
HEADLESS_RENDER_TIMEOUT_MS	15000	Playwright navigation
SERPAPI_PACE_SECONDS	0.4	Between SerpAPI calls
FETCH_CACHE_TTL / failure TTL	1800 / 120	Seconds
DEFAULT_WEB_POLICY	auto	auto required off
DEFAULT_DOMAIN_MODE	general_research	
RUN_CANARY_ON_STARTUP	true	Brain tool-call probe
SEC_EDGAR_USER_AGENT	LocalEngineInternet/1.0 + site	Required by EDGAR
CORS_ALLOWED_ORIGINS	empty	Off until set

16.1 Brain flags (hard prerequisite)

```
--tokenizer-mode deepseek_v4
--tool-call-parser deepseek_v4
--enable-auto-tool-choice
--reasoning-parser deepseek_v4
```

ai-router's README today launches vLLM without these. Until they are set, Mode 2 cannot work and the canary stays false.

16.2 Docker

python:3.12-slim, poppler-utils (pdftotext), Playwright Chromium with OS deps, uvicorn on 8090.
Optional compose profile searxng.

16.3 Metrics

engine_requests_total, engine_active_requests, engine_request_latency_seconds,
engine_search_total{provider,status}, engine_fetch_total{tier,status}, engine_tool_rounds_total.

17. Wiring other repos (documented, not executed here)

1. **phoenician-intelligence** `call_deepseek.py:72` — stop hardcoding DeepSeek’s cloud URL; read `DEEPSEEK_BASE_URL`; point at this engine. Cheap `fetch_company_web_data` shim must set `force_web_search=True` (maps to policy required). Today the `prefetch` shim does not force — that is a bug to fix when wiring.
2. **ai-router** `web_search.py` — replace the body with `POST /v1/search`; delete the knowledge fallback.
3. **phoenician-portfolio** `strategy/deepseek.py` — stop `del tools`; pass `phoenician_tools` when the provider is local. Universe runs cannot search today.
4. **Earnings / EP / PI scrapers** — optional host swap from `serpapi.com/search` to this engine’s `GET /search`.
5. **screening-engine** — local path needs a `complete_with_search` equivalent via this loop (Anthropic-native today).
6. **capiq-screen-agent** — DeepSeek desk must not fall back to Anthropic for web; call this engine.
7. **Brain launch** — add the four vLLM flags.

Do not route `deepseek-v4-pro` to a Flash-only Brain. Do not invent a new protocol if OpenAI chat-completions plus `phoenician_tools` will work.

18. Tests and verification

43 tests, pytest-asyncio + respx, no live keys. Autouse fixture clears every provider key, disables the canary, sets SerpAPI pace to 0, and clears the fetch cache.

File	What it locks in
test_query.py	Legal-suffix strip, site:/filetype: builders, Earnings query list
test_serpapi_parse.py	Fixture JSON: organics, answer_box, KG attributes, only 2 related, intra-payload dedup
test_merge.py	Canonical URL + snippet dedup; ir_discovery drops SEC/Bloomberg; earnings_doc keeps SEC; general keeps Reuters; all-fail raises fail-closed; one provider fail is skipped
test_extract.py	html_to_text unescape/chrome strip; Trafilatura-empty fallback; max_chars
test_dsml.py	Closed invoke, open invoke, tool__calls markup, prose → []
test_budget.py	Use caps; oldest large tool output compacted; URLs retained
test_ladder.py	403 → curl_cffi; httpx_only does not call cffi; thin/IR → Playwright; PDF magic → parser; SEC wrapper follows EX-99.1; social skipped
test_loop.py	Tool round then final; DSML recovered; required re-ask then 424; required then success + both citation forms; policy off is pass-through
test_fail_closed.py	Never “own knowledge”; budget ERROR
test_compat.py	organic_results / answer_box / knowledge_graph / related_questions shape
test_api.py	/health, /capabilities flags, stream+tools 400, bearer and api_key auth

scripts/smoke.py (live): dated factual question with policy required must produce web_search or search_and_read in the trace and a real http URL in sources; policy off must produce no trace. Warns if the canary is false.

19. What this engine deliberately does not do

- Replace ai-router (inference, load balance, Brain keys)
- Call Anthropic, Perplexity, or Gemini as a search backend
- Require LangChain / LangGraph
- Replace CapIQ (Selenium, credentials), AlphaSense, Vertex / Gemini File Search, OpenAI embeddings / pgvector, FMP / yfinance, OpenAI TTS, investor-portal PDF extract, Linker qualitative score
- Hide sources or pre-rank “the” answer
- Tell the model to hallucinate when search is down
- Stream tool calls (known DSML leak)
- Port Earnings’ DeepSeek ranker or 37-query templates
- Auto-include Google CSE in every fan-out
- Wire the other product repos (documented only)

20. File-by-file method index

Path	Responsibility
engine/main.py	Lifespan: shared httpx client, FetchCache, optional canary, mount routers
engine/config.py	All env, enums, VLLM_REQUIRED_FLAGS
engine/auth.py	Bearer or query api_key
engine/errors.py	Fail-closed wording
engine/runtime.py	Process handles
engine/metrics.py	Prometheus
engine/budget.py	ToolBudget + compaction (re-exported as agent/budget.py to avoid import cycles)
engine/search/base.py	Hit, SearchOutcome, canonical_url
engine/search/*.py	One file per provider + merge + query
engine/fetch/policy.py	Deny lists, WAF markers, IR/SEC predicates, Chrome headers
engine/fetch/cache.py	Thread-safe TTL cache
engine/fetch/urlutil.py	href resolve, link extract, pdf URL, title/canonical
engine/fetch/extract.py	Trafilatura + html_to_text
engine/fetch/pdf.py	Five-parser chain
engine/fetch/sec.py	Exhibit discovery
engine/fetch/js_interact.py	Playwright IR driving
engine/fetch/ladder.py	Tier selection
engine/tools/schemas.py	OpenAI function schemas
engine/tools/execute.py	Dispatch + source collection
engine/tools/dsml_fallback.py	Markup → tool_calls
engine/agent/upstream.py	URL join, header drop, usage sum
engine/agent/prompts.py	PI web system + force-search nudge
engine/agent/citations.py	Both PI blocks
engine/agent/loop.py	Mode 2 + canary
engine/api/chat.py	Completions + stream reject/proxy
engine/api/primitives.py	Mode 1 JSON API
engine/api/compat.py	GET /search
engine/api/health.py	health / metrics / capabilities

21. How to run it

```
python3.12 -m venv .venv
source .venv/bin/activate
pip install -r requirements-dev.txt
playwright install chromium          # only if FETCH_MODE=full
cp .env.example .env                # set UPSTREAM_LLM_BASE_URL + one search key
uvicorn engine.main:app --host 0.0.0.0 --port 8090

# or
docker compose up --build
docker compose --profile searxng up  # optional

pytest
ENGINE_URL=http://127.0.0.1:8090 python scripts/smoke.py
```

Mode 2 example (apps already speak this):

```
POST /v1/chat/completions
{
  "model": "deepseek-v4-flash",
  "messages": [{"role": "user", "content": "What was Acme's last reported revenue? Cite a live URL."}],
  "phoenician_web": {"policy": "required", "domain_mode": "general_research"}}
}
```

Mode 1 example (Earnings-style, no LLM in this hop):

```
POST /v1/search
{ "query": "\"Acme\" \"2026\" filetype:pdf", "providers": ["serpapi"], "domain_mode": "earnings_doc"
}

POST /v1/fetch
{ "url": "https://ir.acme.com/q2.pdf", "mode": "full", "domain_mode": "earnings_doc" }
```

Definition of done on a real machine

The code is in GitHub. The method is this paper. The engine is not operationally done until GET /capabilities shows `brain_tool_calls_supported: true` on the actual Brain, at least one search key is set, and smoke.py passes a dated factual question with a real URL in `phoenician_sources`.